

MATH 6241 – Stochastic Processes

❖ Queueing Theory

Instructor: He Wang
Department of Mathematics
Northeastern University

Why queues?

Queues can arise in any system when demand exceeds supply



or when arrival rate exceeds service rate.



Large unexpected queues can be a logistical nightmare, and can cause ripple effects across the economy:



Queueing models appear in:

- Call centers (customers waiting for agents)
- Cloud computing (jobs waiting for CPUs/GPUs)
- Healthcare (patients waiting for treatment)
- Transportation (traffic, toll booths)
- Communication networks (packet queues)

Why mathematical theory?

Design systems to prevent them from arising.

Need to be able to predict how parameter choices in your system will affect the appearance and growth of large queues .

Accurate mathematical modeling can provide this capability.

The model will depend on various parameters, including the

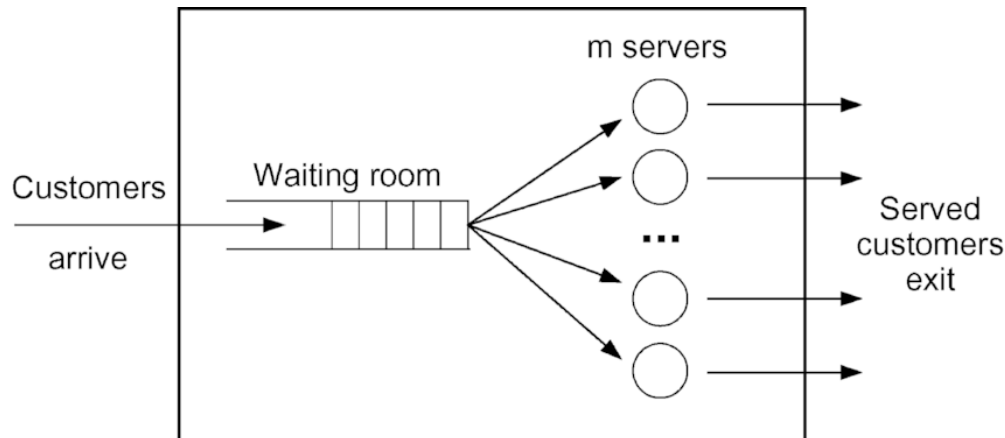
- **Arrival rate** (estimate based on data or reasonable model)
- **Service rate** (number of servers, rate per server, queueing protocol).

Things you typically can't control include:

- Arrival rate.
- Other exogenous variables: queue capacity; cost of service to supplier; etc.

Things you typically can control include:

- Number of servers.
- Queueing discipline (e.g. single queue [think RMV] or separate queues [think supermarket]).
- Service rate (sometimes).



Mathematical modeling

For each choice of parameters, your model should provide information about

- Stability region.
- Average quantities: waiting time, length of queue.
- Unhappy events: probability of large queue, probability of long wait.

Goals:

- Determine how solutions change as model parameters change.
- Design system to mitigate effects of queues.

Note: a queue is a nonlinear system $\Rightarrow$ sensitive to control.

What is Queueing Theory?

Queueing theory studies **random congestion**:

Jobs arrive randomly → wait → receive service → depart.

A *queue* is not just a line — it is a **stochastic dynamical system** driven by randomness in:

- Arrival times
- Service times
- Server availability

❑ Components of a Queue (Modeling Language)

1. Arrival Process

Describes **when customers/jobs arrive**.

- Arrival times: $T_1, T_2, \dots$
- Interarrival times:

$$X_n = T_n - T_{n-1}$$

Common models:

- **Poisson process** $\rightarrow$ exponential interarrivals
- Renewal process $\rightarrow$ general interarrivals
- Time-varying arrivals (nonhomogeneous Poisson)

2. Service Process

Describes **how long service takes**.

Service times: $S_1, S_2, \dots$, Can be:

- Exponential (memoryless)
- Deterministic

Service-time variability is *crucial* for delay.

3. Number of Servers

- Single server (bank teller)
- Multiple servers (call center)
- Parallel identical servers assumed unless stated otherwise

4. Queue Discipline

Rules determining **who gets served next**:

- FIFO (FCFS): first come, first served
- LIFO (Last-In, First-Out)
- Priority queues
- Processor sharing

5. System Capacity & Population

- Infinite vs finite buffer
- Finite vs infinite customer population

6. Kendall's Notation

Queuing theory uses the Kendall notation to classify the different types of queuing systems, or nodes. Queuing nodes are classified using the notation

$$\mathbf{A/S/c/K/N/D}$$

- **A** is the arrival process (examples are M (Poisson process), D (deterministic) and G (general))
- **S** is the mathematical distribution of the service time (*e.g M (exponential) and G (general)*)
- **c** is the number of servers
- **K** is the capacity of the queue, omitted if unlimited
- **N** is the number of possible customers, omitted if unlimited
- **D** is the queuing discipline, assumed first-in-first-out if omitted

Example:

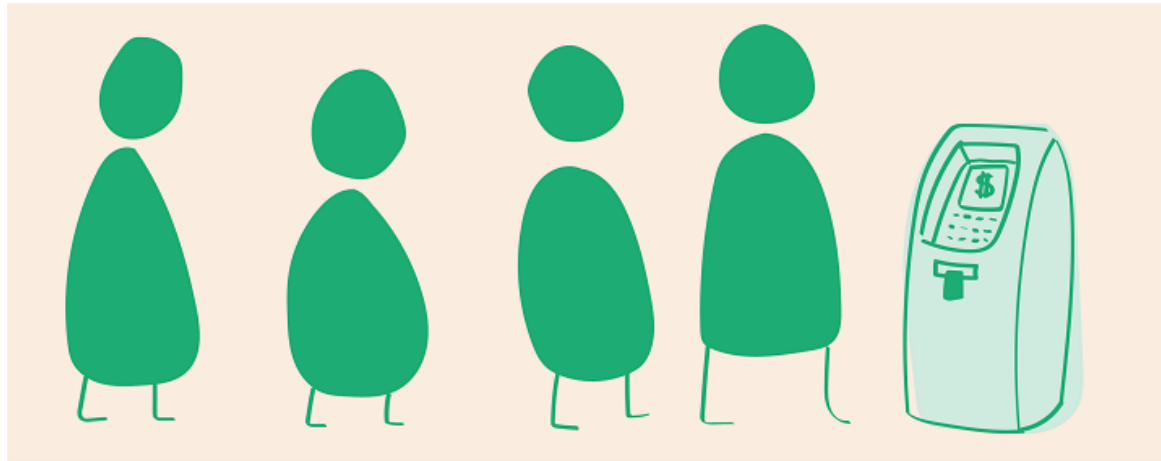
M/M/1 = Poisson arrivals, exponential service, 1 server

M/G/1 = Poisson arrivals, general service

Example: M/M/1 queuing model

For example, think of an ATM. It can serve:

- One customer at a time
- In a first-in-first-out order
- With a randomly-distributed arrival process and service distribution time
- Unlimited queue capacity
- Unlimited number of possible customers.



Queuing theory would describe this system as a M/M/1 queuing model (“M” here stands for Markovian, a statistical process to describe randomness).

7. Poisson Arrival Process (Key Model)

Definition:

$$P(N(t) = k) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}$$

Properties:

- Independent increments
- Stationary increments
- Memoryless interarrival times

Interarrival times: $X_n \sim \text{Exp}(\lambda)$

8. Service Time Models

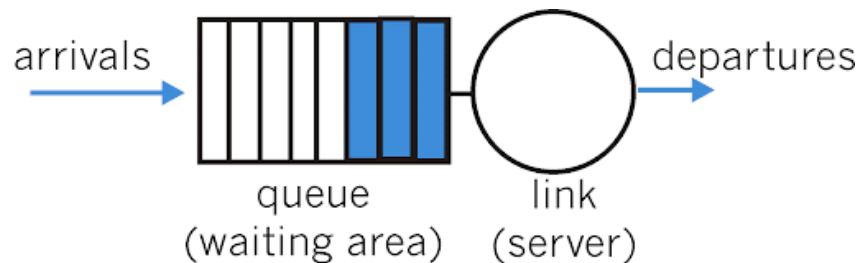
Model	Meaning
Exponential	No memory (M/M/1)
Deterministic	Fixed processing time
General	Arbitrary distribution

Memoryless property: $P(S > t + s \mid S > t) = P(S > s)$

□ Basic assumptions

- The arrivals constitute a renewal process, so the interarrival times X_n are IID.
- The service times S_n of customers are IID.
- Service discipline is FIFO (= First-In, First-Out).

Initially we will consider a single queue with one server.



□ Properties

Let $N(t)$ = number of customer arrivals up to time t

Two important numbers characterize the long-term behavior of the queue.

- $\lambda = \lim_{t \rightarrow \infty} \frac{N(t)}{t} = \text{long-term average arrival rate}$
- $\mu = \text{long-term service rate}$

Units of λ and μ are $(\text{time})^{-1}$.

Service is really described by the average service time for customers:

- $\frac{1}{\lambda} = E[X] = \text{mean time between arrivals}$
- $\frac{1}{\mu} = E[S] = \text{mean service time} = \lim_{N \rightarrow \infty} \frac{1}{N} (S_1 + \cdots + S_N)$

S_i = service time of i -th customer

□ Stability & Traffic Intensity

The ratio of arrival rate and service rate determines whether the queue will remain bounded and stable.

$$\rho := \frac{\lambda}{\mu} = \frac{E[S]}{E[X]} = \text{Traffic density}$$

- System is **stable** if $\rho < 1$
- System is **unstable** (explodes) if $\rho \geq 1$

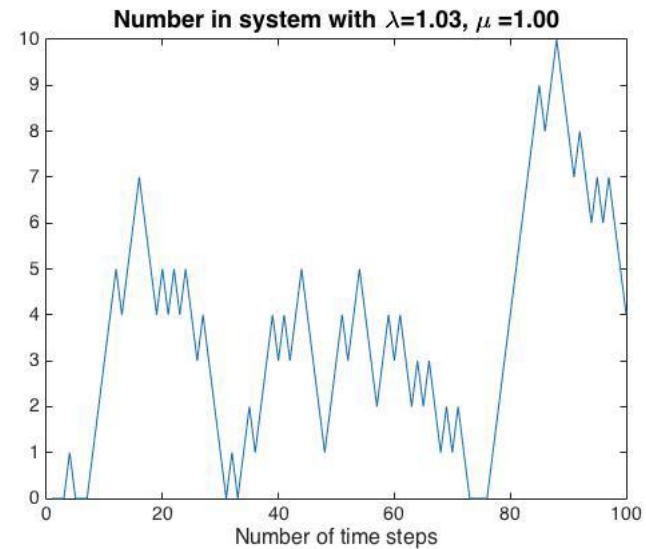
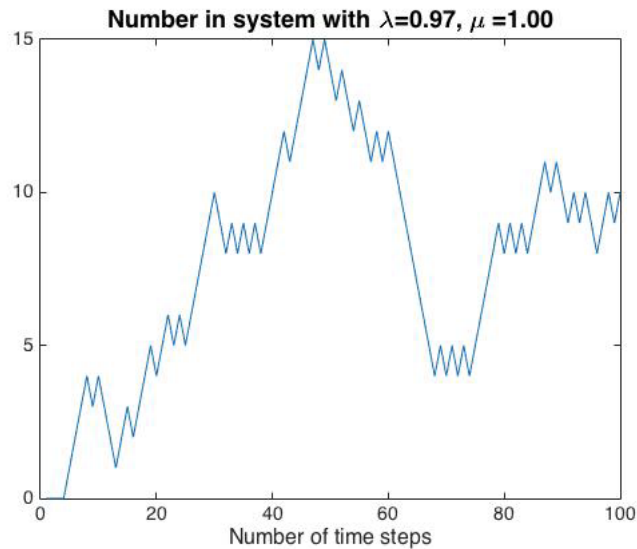
If $\rho > 1$, then new customers arrive faster than they can be serviced, and the length of the queue grows without bound.

$$\lim_{t \rightarrow \infty} L(t) = \infty$$

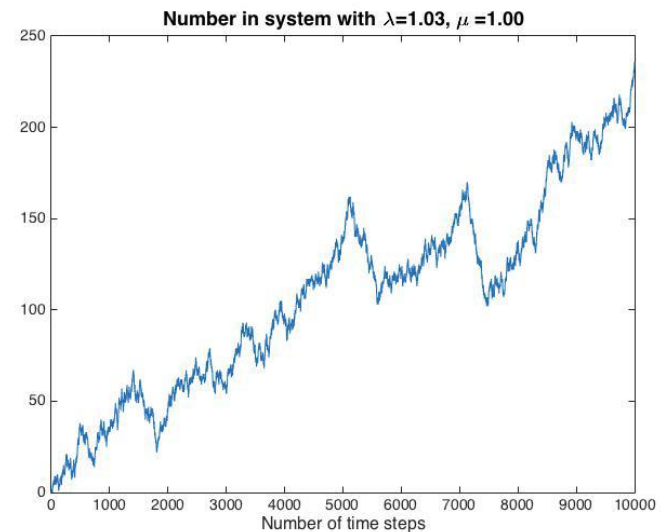
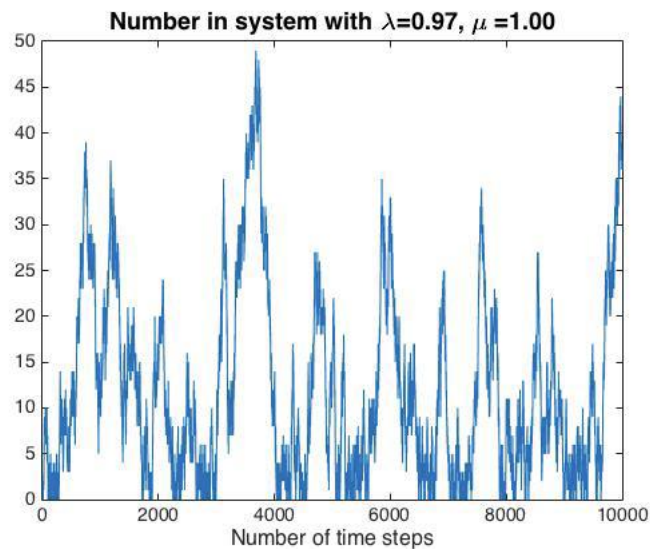
$L(t)$ = number of customers in system at time t

☐ Simulations:

For short times we don't see much difference.



For longer times the difference becomes apparent.



□ Things we want to know

- The probability distribution of the number of customers in the system:
 $\mathbb{P}(L(t) = n)$ (computed in the stationary distribution)
- Utilization of the server = fraction of time the server is busy
- System throughput = rate at which customers leave the system
- Probability distribution of customer waiting time spent in queue.

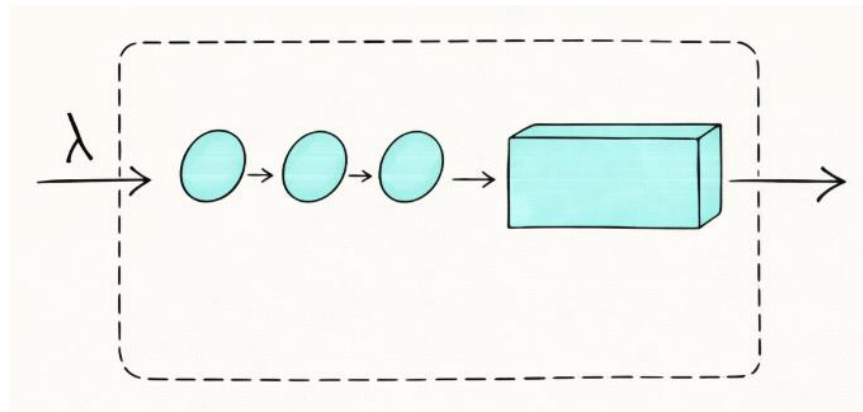
□ Little's Law

Theorem (Little's Law): In a queueing system (queue + service), let

- l denote the long-term **average number** of customers in the system,
- λ the **rate of arrivals**, and
- w the long-term **average time** that a **customer** is in the system.

Then,

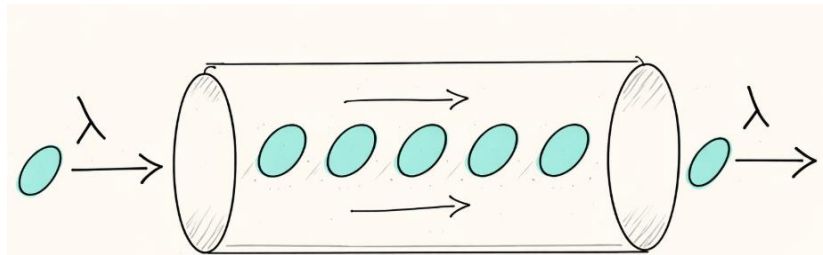
$$l = \lambda w$$



Comments:

This should be understood as a relation between long-run average quantities for a stable system.

It applies to *almost any* queueing system, regardless of arrival process, service-time distribution, service order, or number of servers—provided the system is stable.



l = number in system

Stable $\Rightarrow l \lambda^{-1}$ = time spent in system

Relates two useful quantities but does not predict the value of either.

In order to compute either l or w we need a detailed model.

Little's Formula — Proof by Picture

Rather than proving the result rigorously, we justify it **intuitively** using a *proof by picture*, following the approach of **Gross et al. (2008)**.

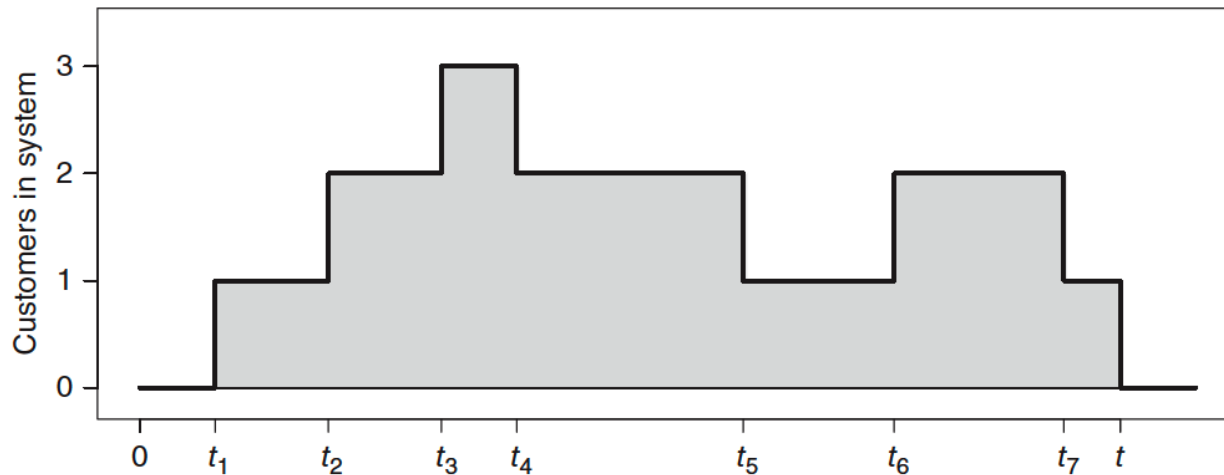
Consider a realization of a queueing system over the time interval $[0, t]$, starting when the **first customer enters** the system and ending when the system is **empty again**.

Suppose **four customers** arrive during $[0, t]$, with arrival and departure times:

Customer	Arrival time	Departure time
1	(t_1)	(t_5)
2	(t_2)	(t_4)
3	(t_3)	(t_7)
4	(t_6)	(t)

Let $N(s)$ denote the **number of customers in the system** at time s .

The sample path of $N(s)$ is a **step function**, increasing by 1 at each arrival and decreasing by 1 at each departure



Let A be the **area of the shaded region** under the curve $N(s)$.

$$A = \int_0^t N(s) ds$$

Little's formula is obtained by finding the area of the shaded region two ways.

- **First Way:** Sum of Customer Sojourn Times (Time View)

Each customer contributes a rectangle of height 1 and width equal to the time they spend in the system.

$$\begin{aligned} A &= (t_5 - t_1) + (t_4 - t_2) + (t_7 - t_3) + (t - t_6) \\ &= (t - t_1) + (t_5 - t_2) + (t_7 - t_6) + (t_4 - t_3) \end{aligned}$$

The **average time spent in the system per customer**, denoted W , is

$$W = \frac{A}{\text{Number of customers in } [0, t]}$$

- **Second Way:** Time Average of System Size (Space View)

Alternatively, compute the area by summing rectangles over **time intervals with constant system size**.

$$A = 1(t_2 - t_1) + 2(t_3 - t_2) + 3(t_4 - t_3) + 2(t_5 - t_4) + 1(t_6 - t_5) + 2(t_7 - t_6) + 1(t - t_7).$$

Dividing by total time t gives the **time-average number of customers in the system**:

$$L = \frac{1}{t} \int_0^t N(s) ds = \frac{A}{t}$$

Equating the Two Views

$$L = \frac{A}{t} = \frac{W \times (\text{Number of customers in } [0, t])}{t}$$

Letting $t \rightarrow \infty$, the **average arrival rate** is

$$\lambda = \lim_{t \rightarrow \infty} \frac{\text{Number of customers in } [0, t]}{t}$$

Hence

$$L = \lambda W$$

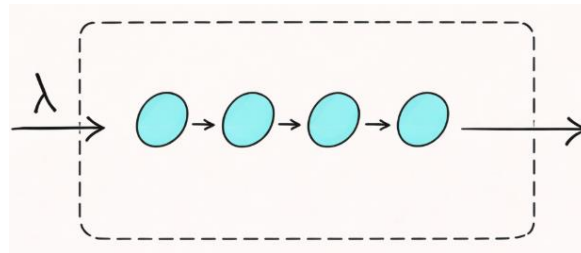
Applications of Little's Law

The 'system' in Little's Law is very broad, and can refer to the whole system of queue and server, or just the queue by itself, just the server by itself, or subsystems of a queueing network.

Application 1. Queue only.

- The arrival rate is λ
- l^Q = average number of customers in **queue**
- w^Q = mean time spent waiting in **queue** by customer

$$l^Q = \lambda w^Q$$



Application 2. Server only.

Suppose we have a single server.

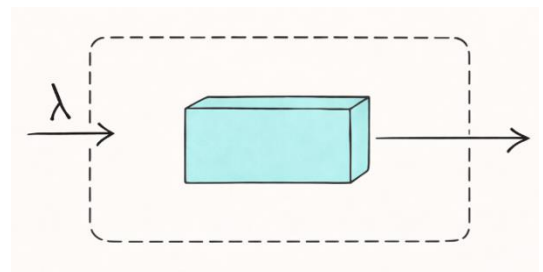
- The number in the server is either 0 or 1.
- Then l^S is the **fraction of time the server is busy**.

The mean time spent in the server is $w^S = \mu^{-1}$, the mean service time.

- If $\rho < 1$, the server can keep up with arrivals, so jobs enter service at rate λ
- If $\rho \geq 1$, the server is always busy, and jobs enter service at the **maximum possible rate μ**

So, The arrival rate to the server is

$$\lambda^S = \begin{cases} \lambda & \text{if } \rho < 1 \\ \mu & \text{if } \rho \geq 1 \end{cases}$$



Applying Little's Law to the Server

$$l^S = \lambda^S w^S$$

Substituting $w^S = \mu^{-1}$. So,

$$l^S = \begin{cases} \frac{\lambda}{\mu} = \rho & \text{if } \rho < 1 \\ 1 & \text{if } \rho \geq 1 \end{cases}$$

So Little's Law gives

$$l^S = \text{fraction of busy time for server} = \min\{\rho, 1\}$$

- If $\rho < 1$, the server is busy a fraction ρ of the time.
- If $\rho \geq 1$, the server is busy **all the time**.

Application 3. Infinite Server. ($M/M/\infty$ intuition)

Suppose we have infinitely many servers, all working at rate μ .

- **No waiting:** every arriving customer immediately enters service.
- The *queue* is always empty.
- The system size equals the number of customers **in service**.

Let

λ = arrival rate

μ = service rate per server

l = mean number of customers being served (i.e., in the system)

Since there is no waiting,

$$w = \text{mean service time} = \mu^{-1}.$$

Little's Law holds for this system because it is stable

$$l = \lambda w = \frac{\lambda}{\mu} = \rho$$

In the infinite-server model, ρ simply equals the average number of customers in se

Application 4. Bitcoin — Little's Law in the Wild

When you initiate a Bitcoin transaction (Tx) from your wallet, it does **not** immediately enter the blockchain. Instead, it is first sent to a waiting area called the **mempool**. At some later time, a miner includes the transaction in a block; once that block is added to the blockchain, the transaction is confirmed.

From a queueing perspective:

- **Arrivals**: new transactions broadcast to the network
- **System**: the mempool
- **Departures**: transactions included in mined blocks

Bitcoin Mempool as a Queue

Data (2022): $\lambda \approx 2$ transactions/second.

Average number of transactions waiting in the mempool: $l = 18,000$

Applying Little's Law $l = \lambda w$

$$w = \frac{l}{\lambda} = \frac{18000}{2} = 9,000 \text{ seconds} \approx 2.5 \text{ hours.}$$

Updated Snapshot (Jan. 5, 2026)

$$\lambda \approx 10.77 \text{ transactions/second.}$$

$$l \approx 40,248$$

$$w = \frac{l}{\lambda} = \frac{40,248}{10.77} = 3738 \text{ seconds} = 1.04 \text{ hours}$$

Average time for a transaction to be included in a block is around 60 minutes

See live updates: https://colab.research.google.com/drive/1AL9mmh-JYb5Bs3IrrkliISL_gSLtV4MQ?usp=sharing

Example (Carwash)

Cars arrive at a drive-through carwash according to a **Poisson process** at a rate of

$$\lambda = 9 \text{ cars per hour.}$$

The time required to wash a car is **exponentially distributed** with mean **5 minutes**, i.e.,

$$\text{mean service time} = \frac{1}{12} \text{ hour.}$$

Thus, the **service rate** is

$$\mu = 12 \text{ cars per hour.}$$

We address the following questions:

- How many cars, on average, are at the carwash?
- How long, on average, is a customer at the carwash?
- How long, on average, does a customer wait to be served?
- What is the expected number of cars waiting to be served?

Queueing Model and Basic Properties

Let X_t denote the number of cars in the system at time t .

Because arrivals are Poisson and service times are exponential with a single server, the system is an $M/M/1$ **queue**.

This is a **birth–death process** with:

- birth (arrival) rate $\lambda = 9$,
- death (service completion) rate $\mu = 12$.

The **traffic intensity** is $\rho = \frac{\lambda}{\mu} = \frac{9}{12} < 1$. so the system is **stable** and has a limiting (steady-state) distribution.

For an $M/M/1$ queue with $\rho < 1$, the **stationary probabilities** are

$$\pi_k = (1 - \rho)\rho^k, k = 0, 1, 2, \dots$$

Substituting $\rho = 3/4$,

$$\pi_k = \left(1 - \frac{3}{4}\right) \left(\frac{3}{4}\right)^k = \frac{1}{4} \left(\frac{3}{4}\right)^k, k = 0, 1, 2, \dots$$

Thus, the number of cars in the system has a **geometric distribution** on $\{0, 1, 2, \dots\}$ with parameter

$$p = 1 - \rho = \frac{1}{4}.$$

1. Average Number of Cars at the Carwash

The mean of a geometric distribution on $\{0, 1, 2, \dots\}$ with parameter p is

$$\frac{1 - p}{p}$$

Here
$$E[X] = \frac{1}{1 - \rho} = \frac{\frac{\lambda}{\mu}}{1 - \frac{\lambda}{\mu}} = \frac{\lambda}{\mu - \lambda}$$

$$\text{So, } L = \frac{9}{12 - 9} = 3$$

2. Average Time a Customer Spends at the Carwash

Let:

- L = long-run average number of cars in the system,
- W = long-run average time a customer spends in the system.

By **Little's Law**, $L = \lambda W$, so,

$$W = \frac{L}{\lambda} = \frac{3}{9} = \frac{1}{3} \text{ hour} = 20 \text{ minutes}$$

3. Average Waiting Time Before Service

Let:

- W_q = average time spent **waiting in the queue**,
- W_s = average **service time**.

$$W = W_q + W_s.$$

Since service times are exponential with rate μ ,

$$W_s = \frac{1}{\mu} = \frac{1}{12} \text{ hour} = 5 \text{ minutes}$$

$$W_q = W - W_s = \frac{1}{3} - \frac{1}{12} = \frac{1}{4} \text{ hour} = 15 \text{ minutes.}$$

4. Expected Number of Cars Waiting to Be Served

Now we consider **only the queue** (excluding the server)

Let:

- L_q = long-run average number of cars waiting in the queue.

Little's Law applies to the queue alone:

$$L_q = \lambda W_q = 9 \times \frac{1}{4} = 2.25 \text{ cars waiting in the queue.}$$

Little's Law is truly remarkable.

It is applicable almost completely independent of the details of the system.

As it doesn't depend on detail, unsurprisingly it cannot provide detail.

What follows is in contrast to that. **Lindley's Equation**